Neurocomputing 648 (2025) 130578

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

DSAIS-PINN: Dynamic seeds allocation importance sampling for physics-informed neural networks

Wentao Feng, Ming Hu, Rui Huang, Chenwei Tang, Shudong Huang *, Jiancheng Lv

College of Computer Science, Sichuan University, No. 24 South Section 1, Yihuan Road, Chengdu, 610065, Sichuan Province, PR China

ARTICLE INFO

Communicated by Q. Ni

Keywords:
Physics-informed neural networks
Voronoi tessellation
Interpolation

ABSTRACT

Physics-Informed Neural Networks (PINNs) have gained widespread recognition as a capable tool for solving partial differential equations (PDEs), achieving notable success across various applications. Conventional PINNs, on the other hand, encounter significant challenges, including slow convergence rates and substantial computational costs. Recent advancements have incorporated importance sampling methods into the training process of PINNs, leading to considerable improvements in convergence speed and overall efficiency. Despite these advances, current importance sampling techniques typically calculate precise sampling probabilities for seed points and interpolate for non-seed points, which may lead to inaccuracies in high-loss regions. This study introduces the Dynamic Seed Allocation Importance Sampling (DSAIS) method, which dynamically adjusts the number of seed points in regions, i.e., Voronoi cells, with varying loss levels, prioritizing high-loss areas. This refinement improves interpolation accuracy for non-seed points. The proposed method's effectiveness is confirmed through its application to three benchmark issues: the Schrödinger equation, the 1D Burgers equation, the Korteweg–de Vries equation, and the 2D heat equation. Performance is assessed using standard evaluation metrics, including maximum error, mean absolute error, and root mean square error. The outcomes demonstrate that DSAIS achieves state-of-the-art performance. DSAIS shows up to 63%, 30%, 30% smaller root mean square error in Schrodinger equation, Burgers equation and KDV equation, respectively.

1. Introduction

Partial Differential Equations (PDEs) are indispensable in natural sciences, engineering, and social sciences and have attracted significant attention from researchers due to their extensive applications in fields such as physics, chemistry, mathematical biology, dynamical systems, and engineering [1]. As a mathematical tool, they provide precise descriptions of changes in continuous media. PDEs serve as the theoretical foundation for studying physical phenomena. For example, in heat conduction, the heat equation describes the variation of temperature over time and space [2]; in fluid mechanics, the Navier–Stokes equations reveal fluid motion laws [3]. In the field of epidemiology, Sher et al. [4] investigated the computational and theoretical modeling of novel COVID-19 transmission dynamics under the Mittag-Leffler power law [5]. However, due to the complexity of geometries, high dimensionality, or nonlinear characteristics in many practical issues, analytical solutions to PDEs are often difficult or impossible. Consequently, numerical methods such as finite difference [6], finite volume [7], and spectral methods [8] have become the primary tools for solving PDEs [9]. While traditional numerical methods are known for their high accuracy, they encounter several significant challenges. First, they suffer from the curse of dimensionality, where the number of grid points increases exponentially as the number of dimensions grows [10]. Second, solving complex PDEs often requires multiple iterations to achieve convergence, leading to substantial computational costs [11]. Third, implementing these classical methods is inherently complex, often involving millions of lines of code, making them difficult to maintain and update over time.

With the rapid advancements in GPU-accelerated computing, deep learning (DL) has made significant progress in fields such as computer vision (CV), natural language processing (NLP), and automatic speech recognition (ASR). However, its application to scientific computing (SC) has remained relatively underexplored for a considerable period. According to the universal approximation theorem [12], no matter how complex a function is, it can be approximated by a neural network as long as the network is sufficiently large. This opens up the possibility of using neural networks to solve complex partial differential equations (PDEs) [13] that govern physical phenomena such as turbulence, heat transfer, and fluid–structure interactions.

In 2019, the Crunch group at Brown University introduced a pioneering computational physics framework known as Physics-Informed

* Corresponding author.
E-mail address: huangsd@scu.edu.cn (S. Huang).

https://doi.org/10.1016/j.neucom.2025.130578
Received 22 January 2025; Received in revised form 7 May 2025; Accepted 21 May 2025
Available online 6 June 2025
0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Neural Networks (PINNs) [14], which utilizes neural networks to solve PDEs directly. This innovative methodology has gained considerable attention and has been applied across a wide range of disciplines that involve differential equations [15]. For example, in the work by Han et al. [16], a physics-informed neural network (PINN) based on the SIR model was employed. Despite discrepancies between the theoretical model and actual epidemic dynamics (given the differences between SAIRD and SIR), it accurately identified and predicted transmission trends from real-world data, showcasing the high potential and effectiveness of combining PINNs with compartmental models in epidemiology. Additionally, Hu et al. [17] proposed an enhanced PINN that incorporates wavelet transforms and a novel loss function, enabling efficient, stable, and accurate estimation of both the unknown number of infected individuals and the parameters in the COVID-19 transmission model, even in scenarios where these parameters vary over time.

PINNs represent a novel integration of scientific computing with traditional numerical analysis, effectively tackling various PDE-related challenges, including equation solving, parameter inversion, model discovery, control, and optimization. Compared to conventional numerical methods, PINNs offer several distinct advantages. They provide approximate, mesh-free solutions with controllable analytical gradients and allow for the simultaneous execution of both forward and inverse modeling within a unified optimization framework. Despite these successes, PINNs still face several limitations: (1) While effective for short-term time integration, their application to long-term transient PDEs becomes computationally expensive as the spatiotemporal domain grows arbitrarily large [18]; (2) Training PINNs requires sampling a large number of points from initial conditions (ICs), boundary conditions (BCs), and the domain, often necessitating numerical methods to compute the true solution. The diverse training dynamics of these different types of points further complicate the training process [19]; (3) The reliance on automatic differentiation results in low computational efficiency and slow convergence rates. To address these challenges, researchers have explored various enhancements to PINNs [20,21]. Notably, importance sampling methods have been introduced to improve convergence speed and computational efficiency, marking a significant step forward in overcoming these limitations.

Although previous importance sampling methods have improved the convergence speed of PINNs, they provided limited improvements in solution accuracy. Since regions with higher loss naturally deserve more attention, existing importance sampling methods typically calculate the sampling weights or probabilities only for seed points accurately and then use interpolation or other methods to determine the weights for non-seed points. This approach may result in less accurate interpolation for some non-seed points in high-loss regions. By dynamically adjusting the number of seed points in regions with varying loss levels, allocating more seed points to high-loss regions, and subsequently using these seed points to interpolate the weights of other non-seed points, the accuracy of interpolation for non-seed points can be significantly improved. To address this gap, this paper introduces a novel approach: the Dynamic Seed Allocation Importance Sampling (DSAIS) method. This approach aims to improve solution accuracy while maintaining efficient convergence. Importance sampling accelerates convergence by prioritizing key regions of the solution domain, but it typically requires calculating the sampling probability for each point, which results in substantial computational overhead [21]. The proposed DSAIS method addresses this challenge by dynamically allocating seed points, allowing for a more accurate estimation of their sampling probabilities. These probabilities are then propagated to other points in the domain through interpolation, effectively balancing computational efficiency with enhanced accuracy.

Our contributions can be summarized as follows:

- A Voronoi diagram is employed to partition the space into distinct regions, facilitating the application of importance sampling within each cell. This technique helps mitigate the risk of oversampling in regions with high error and undersampling in areas with low error, ensuring a more balanced and representative sampling process. The partitioning enables a more efficient allocation of resources, contributing to more accurate model predictions.
- We propose an improved importance sampling method that dynamically adjusts the allocation of seed points to the cells based on their respective loss values. This adaptive approach allows the model to prioritize cells with higher loss values, focusing computational resources on regions where the model's performance is weaker. As a result, the method effectively directs attention to areas that are more critical for model improvement, leading to better overall accuracy and performance.
- The effectiveness and practicality of the proposed method are thoroughly evaluated through a series of experiments, which highlight its ability to significantly improve both sampling efficiency and the overall performance of the model. The experimental results confirm that the proposed approach leads to enhanced generalization, faster convergence, and more reliable predictions in comparison to traditional methods.

2. Related work

To provide clarity, this section summarizes foundational concepts and methods relevant to the study. Section 2.1 explains the fundamental principles, challenges of PINNs and their variants, including their loss formulation and optimization strategies. Section 2.2 highlights the importance sampling methodology, focusing on its integration into training frameworks to improve computational efficiency and convergence.

2.1. Physics-informed neural networks

The core concept of PINNs is illustrated in Fig. 1. The primary goal of this novel framework is to develop a neural network model that serves as a surrogate for the solution $u(x)$ of given PDEs. By embedding the physical laws governing the system directly into the learning process, PINNs integrate data-driven modeling with the mathematical rigor of physics-based formulations. This integration enables NNs to approximate the solution and inherently satisfy the constraints imposed by the underlying PDEs, ICs, and BCs. A general formulation, applicable to a wide range of PDEs, is expressed as follows:

$$\begin{aligned} \mathcal{N}[u](x,t) &= f(x,t), && x \in \Omega,\ t \in [0,T] \\ u(x,0) &= g(x), && x \in \Omega \\ \mathcal{B}[u](x,t) &= h(x,t), && x \in \partial\Omega,\ t \in [0,T] \end{aligned} \tag{1}$$

where $\mathcal{N}[\cdot]$ denotes a linear or non-linear differential operator, and $\mathcal{B}[\cdot]$ represents the differential operator associated with BCs. The initial conditions are denoted as $u(x,0) = g(x)$. Here, x is a D-dimensional position vector, and Ω is a subset of $\mathbb{R}^D$ with boundary $\partial\Omega$.

The solution $u(x,t)$ is approximated using a neural network, where the network parameters (w,b) are optimized by minimizing a composite loss function that incorporates physics constraints, boundary conditions, and observational data. The overall loss function is formulated as:

$$\begin{aligned} \mathcal{L}(\theta) = {} & w_f \mathcal{L}_{PDE}\left(\theta;\mathcal{T}_f\right) + w_i \mathcal{L}_{IC}\left(\theta;\mathcal{T}_i\right) + \\ & w_b \mathcal{L}_{BC}\left(\theta;\mathcal{T}_b\right) + w_d \mathcal{L}_{\text{Data}}\left(\theta;\mathcal{T}_{\text{data}}\right) \end{aligned} \tag{2}$$

where w_f, w_i, w_b, and w_d are weights associated with different loss components. The terms $\mathcal{L}_{PDE}$, $\mathcal{L}_{IC}$, $\mathcal{L}_{BC}$, $\mathcal{L}_{Data}$ correspond to losses for PDE residual, ICs, BCs, and observational data, respectively. Each loss term is typically computed as the mean of squared error (MSE) over a set of collocation points. For instance, the PDE residual loss $\mathcal{L}_{PDE}$ over $\mathcal{T}_f$ collection points is expressed as:

$$\mathcal{L}_{PDE}\left(\theta;\mathcal{T}_f\right) = \frac{1}{\mathcal{T}_f}\sum_{i=1}^{\mathcal{T}_f}\left|\mathcal{N}[u_\theta](x_i,t_i) - f(x_i,t_i)\right|^2 \tag{3}$$

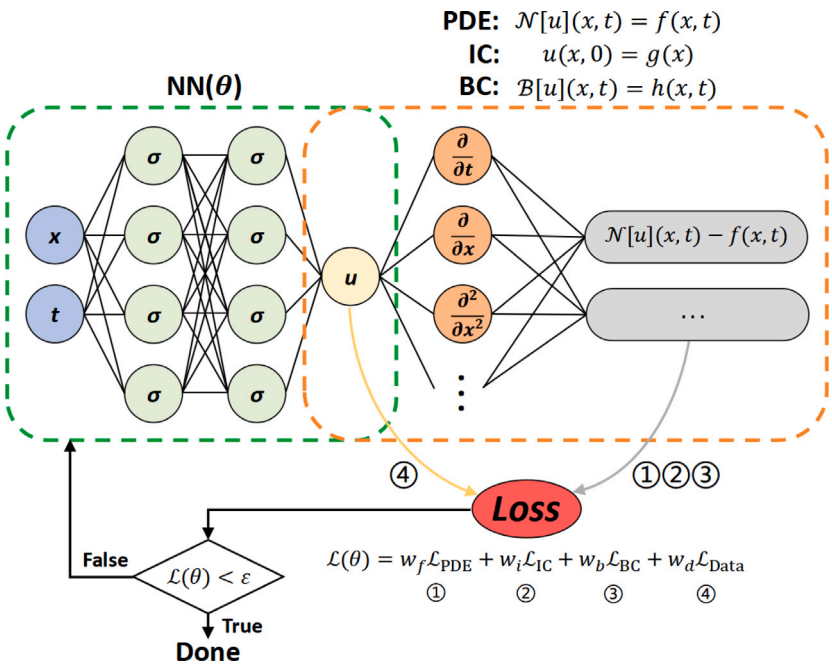


Fig. 1. PINNs network architecture.

Finally, the optimization process in PINNs employs a gradient-based algorithm to minimize the loss function. The network parameters are iteratively updated and refined step-by-step until the desired convergence threshold for the loss function and the target prediction accuracy are achieved, denoted as θ^*.

However, this process is inherently complex and presents significant challenges. One notable difficulty arises from the differing convergence rates of the various loss terms in PINNs. These loss terms may not only converge at different speeds but can also conflict with one another, making it harder for the optimization process to balance all constraints effectively. As a result, the overall performance of PINNs is strongly influenced by the choice of optimization algorithm. In practice, gradient-based methods such as gradient descent, Adam, and L-BFGS are commonly employed to minimize the loss function. Each optimizer comes with its characteristics, and the quasi-Newtonian properties of L-BFGS often make it particularly effective in handling the optimization challenges of PINNs. Therefore, even when the neural network architecture is predetermined, the selection of an appropriate optimizer plays a pivotal role in shaping the quality and reliability of the final solution.

Although PINNs have achieved success in many fields, there remain many strategies for improvement based on the issues mentioned above. In the work of Huang [22] et al. enhancements to PINNs are divided into three categories: improvements to the input, the network, and the loss function. Regarding input improvements, Kovacs et al. [23] modeled the parameters determining the coefficients of the eigenvalue equations as additional inputs. Another approach in this direction is the sampling of input points; Daw et al. [24] proposed an Evolutionary Sampling (Evo) method, which gradually accumulates sample points in regions with high PDE residuals. In Wu et al.'s work [25], a residual-based adaptive sampling method was introduced that not only reduces the number of high-residual points but also improves the solution accuracy for both forward and inverse problems. In terms of network model enhancements, Zhao et al. [26] proposed a new transformer-based architecture—PINNsFormer—that utilizes multi-head attention mechanisms to capture temporal dependencies, thereby approximating PDE solutions more accurately. Furthermore, Gao et al. [27] integrated GANs into PINNs by employing a groupsort activation function in the discriminator, enabling the generator to learn the uncertainty in the PDE solutions observed from the initial and boundary data. In the work of Ren et al. [28], a convolutional long short-term memory (ConvLSTM) network based on an encoder–decoder structure was developed for low-dimensional spatial feature extraction and temporal evolution learning. Concerning loss function optimization, McClenny et al. [29] proposed an adaptive weighting method that assigns greater weight to points with larger losses, while Wang et al. [30] introduced a learning rate annealing algorithm that utilizes gradient statistical information to balance the interplay among different components of the composite loss function. The importance sampling method proposed in this paper is an improvement from the input perspective.

2.2. Importance sampling

Importance sampling is widely regarded as one of the most effective techniques in Monte Carlo simulations [31]. Its core principle involves altering the sampling strategy to focus on regions of the input space that have a greater impact on the simulation outcomes, rather than sampling uniformly or directly from the original probability distribution. By assigning higher probabilities to more critical regions, this approach significantly improves computational efficiency, reduces simulation time, and decreases the variance of the results [32], making it especially valuable in scenarios with high-dimensional or complex distributions. In the context of DL, importance sampling plays a critical role in accelerating the convergence of stochastic gradient descent (SGD) [33]. By prioritizing samples that contribute more substantially to the gradient updates of model parameters, importance sampling ensures that computational resources are directed toward the most informative data points, thereby improving optimization efficiency and model performance. This ability to balance computational effort and impact has made importance sampling a powerful tool in various deep learning applications, particularly in tasks involving large datasets or imbalanced data distributions.

Despite its demonstrated effectiveness, the application of importance sampling to the training of physics-informed neural networks (PINNs) remains relatively unexplored. To date, only a few works, such as those by Nabian et al. [34] and Yang et al. [21], have attempted to leverage importance sampling within the PINNs framework. These studies highlight its potential to address key challenges in PINNs, such as imbalanced error distribution across the domain or inefficiencies in

the optimization process. However, further research is needed to fully explore and refine the integration of importance sampling with PINNs, paving the way for more efficient and robust training methodologies.

3. Methodology

The optimization objective of PINNs is to determine the set of network parameters that minimize the total loss function. As noted above, Eq. (2) has encapsulated the underlying physics, ICs, BCs, and observational data into a unified framework, laying the foundation for achieving the following refinement goal:

$$\theta^* \approx \arg\min_{\theta} \mathcal{L}(\boldsymbol{\theta}) \tag{4}$$

Since the ICs and BCs are treated as penalty terms in the overall loss function, importance sampling is primarily applied to the selection of collocation points $\mathcal{T}_f$. The core idea of importance sampling in this context is to modify the sampling distribution $q(x)$ such that points with greater contributions to the overall loss function are prioritized. By replacing the uniform sampling distribution with this modified one, training samples are drawn more effectively, and computational resources are concentrated on the domain regions most critical to improving the solution.

The modified loss function for PDE residuals incorporating importance sampling can be expressed as:

$$\mathcal{L}_f = \frac{1}{\left|\mathcal{T}_f\right|} \sum_{i=1}^{\left|\mathcal{T}_f\right|} \frac{p_i}{q_i} \ell_f(\mathbf{x}_i; \theta) \tag{5}$$

where $\mathcal{T}_f$ denotes the size of collection points, ℓ_f represents the PDE residual of each point, p_i is the true probability of sampling the point $\mathbf{x}_i$, and q_i is the alternative sampling probability introduced through importance sampling. In the default case of uniform sampling, $p_i = \frac{1}{\mathcal{T}_f}$ for all points. However, importance sampling adjusts q_i to better align with the structure of the loss function, thereby improving training efficiency and convergence.

The key challenge now lies in determining the target distribution q_i for $i \in \{1, 2, \ldots, |\mathcal{T}_f|\}$ that effectively accelerates convergence. To address this, the work of [34] proposed defining the convergence rate $C^{(i)}$ at step i as:

$$C^{(i)} = -\mathbb{E}_f \left[\|\theta^{(i+1)} - \theta^*\|_2^2 - \|\theta^{(i)} - \theta^*\|_2^2\right] \tag{6}$$

where $\theta^{(i)}$ and $\theta^{(i+1)}$ represent the parameters at consecutive optimization steps.

In [35,36], the optimal sampling probability q^* for a collocation point x_i was derived to be proportional to the magnitude of the PDE residual's gradient, i.e., $q^* \propto \|\nabla_\theta \ell_f(\mathbf{x}, \theta)\|_2$. This ensures that points with higher gradients, which substantially impact the loss reduction, are sampled more frequently. However, calculating the gradient of the PDE residual for each collocation point incurs a prohibitively high computational cost, especially for large-scale problems. To overcome this, the work of Katharopoulos and Fleuret [37] demonstrated that directly using the loss function values instead of the gradient is a reasonable and computationally efficient alternative. As a result, the sampling probability q_i can be approximated:

$$q_j^{(i)} \approx \frac{\ell_f(\boldsymbol{\theta}^{(i)}; x_j)}{\sum_{j=1}^{\mathcal{T}_f} \ell_f(\boldsymbol{\theta}^{(i)}; x_j)}, \quad \forall j \in \{1, \ldots, \mathcal{T}_f\} \tag{7}$$

where $q_j^{(i)}$ represents the normalized probability of selecting point $\mathbf{x}_j$ at step i, computed based on its contribution to the total loss.

While Eq. (7) effectively reduces the computational complexity for calculating the sampling probability of each point, it still requires evaluating the loss function at every collocation point, which imposes a considerable computational burden, particularly for large-scale problems. To overcome this limitation, we propose the Dynamic Seed Allocation Importance Sampling (DSAIS) strategy, which interpolates sampling probabilities to significantly lower computational costs while maintaining efficiency and accuracy. The DSAIS approach begins by randomly selecting a small subset of collocation points as initial seed points. These seed points serve as representatives of their respective regions in the computational domain. A Voronoi tessellation is then performed using seed points to partition the entire set of collocation points into distinct cells, each associated with a unique seed point. By utilizing Voronoi's adaptivity, suitability for importance sampling, clear spatial partitioning, and computational efficiency, this approach dynamically adjusts to local problem characteristics. Loss at each seed point is computed as the average loss within its cell, limiting computational costs to these key points rather than the full dataset. This not only enhances accuracy for multi-scale problems but also optimizes training efficiency and reduces the overhead of constructing grids, thereby boosting PINNs' capability to solve complex PDEs efficiently.

To achieve an adaptive and efficient distribution of seed points, cells with varying loss levels are allocated different numbers of additional seed points. This allocation is guided by the sampling probability of the initial seed point in each cell. Specifically, the number of seed points assigned to a given cell is determined by multiplying the sampling probability of the initial seed point by the total number of additional seed points to be distributed. This step ensures that regions with higher loss are given more sampling attention, allowing the model to focus on areas that are more challenging to approximate. Next, additional seed points are randomly selected from the collocation points within each cell based on the assigned number. For instance, if 100 initial seed points are selected, the computational domain is divided into 100 cells. Suppose the goal is to allocate 1000 additional seed points. If the sampling probability of a specific initial seed point is 0.1, then $0.1 * 1000 = 100$ additional seed points will be assigned to that cell. If the computed number of additional seed points exceeds the total number of points in a cell, then all points in that cell are considered as seed points. These newly added seed points are subsequently used to compute sampling probabilities more accurately within their respective cells.

Once the sampling probabilities for the additional seed points are calculated, interpolation is employed to estimate the sampling probabilities for the remaining collocation points within each cell. This interpolation step allows the model to approximate the sampling probabilities efficiently without directly computing the loss for every point, further reducing computational costs. The inverse distance weighting (IDW) method was adopted in this work, which is based on the fundamental assumption that points closer to an unknown point have a greater influence than those farther away. In IDW, the predicted value is calculated as the weighted average of the surrounding known points, with weights determined by the inverse of their distances. For an unknown point p, its predicted value, $\hat{z}(p)$, is computed using the values z_i of the surrounding known points as follows:

$$\hat{z}(p) = \frac{\sum_{i=1}^{N} \omega_i z_i}{\sum_{i=1}^{N} \omega_i}, \tag{8}$$

where N represents the number of known points used for interpolation, and ω_i is the influence weight of the known point on the unknown point p, calculated as:

$$\omega_i = \frac{1}{d_i^{\alpha}}. \tag{9}$$

In practice, we set $\alpha = 2$, which means that the weight decays with the square of the distance. The number of additional seed points is a tunable parameter that can be adjusted depending on the problem's complexity and the desired balance between accuracy and computational efficiency. After assigning sampling weights and probabilities to all points in the domain, the importance sampling method is applied to select mini-batch samples for training. This ensures that points contributing more significantly to the total loss function are prioritized during training, improving the model's ability to focus on critical

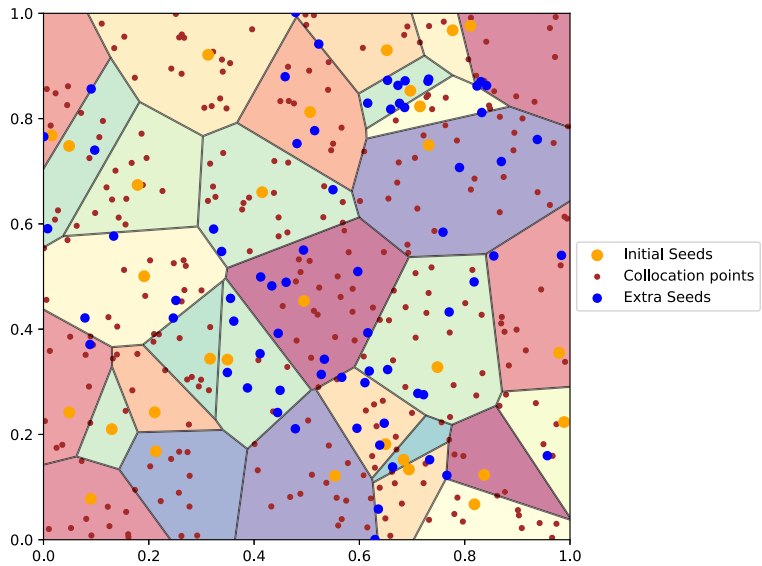


Fig. 2. Voronoi tesselation of collection points, with orange dots representing seed points initially used for partitioning and localization, blue dots representing seed points allocated based on loss value, and brown dots representing collection points.

regions of the solution domain. The entire DSAIS process, including seed selection, Voronoi partitioning, and probability interpolation, is visually illustrated in Fig. 2.

The pseudo-code of a sampler with DSAIS is shown as Algorithm 1.

Algorithm 1 Dynamic Seed-Adaptive Importance Sampling (DSAIS)

Generate $\mathcal{T}_f$ collocation points x_j, sampled from the domain Ω, $j = 1, ..., \mathcal{T}_f$.
Randomly select S initial seed points x_s from x_j, $s = 1, ..., S$.
Perform Voronoi tessellation to partition Ω into S cells, each associated with a seed.
Set model architecture: number of layers, layer dimensions, activation functions, etc.
Specify optimizer hyperparameters: learning rate η, batch size m, and convergence tolerance ϵ.
Initialize model parameters $\theta^{(0)}$.
while $\mathcal{L}(\theta) > \epsilon$ **do**
  Compute the loss value $\mathcal{L}_{x_s}$ for each seed x_s, $s = 1, ..., S$.
  Compute sampling probabilities for initial seeds:

$$q_s = \frac{\mathcal{L}_{x_s}}{\sum_{k=1}^{S} \mathcal{L}_{x_k}}, \quad s = 1, ..., S$$

  Allocate N_{add} additional seed points to cells:

$$N_s = \lfloor q_s \cdot N_{\text{add}} \rfloor, \quad s = 1, ..., S$$

  Randomly select N_s additional seed points within each cell.
  compute sampling probabilities for additional seed points:

$$q_{add} = \frac{\mathcal{L}_{x_j}}{\sum_{k=1}^{N_{\text{add}}+S} \mathcal{L}_{x_k}}, \quad j = 1, ..., N_{\text{add}} + S$$

  Interpolate q_j for all non-seed points in each cell.
  Use importance sampling to select a mini-batch of m points based on sampling probabilities.
  Take a gradient descent step to update θ:

$$\theta^{(i+1)} = \theta^{(i)} - \eta \nabla_\theta \mathcal{L}(\theta^{(i)})$$

end while

4. Experimental setup

The effectiveness of the proposed method is assessed in the next two sections by solving a set of representative PDEs. Section 4 provides a detailed description of the experimental design, including the selected equations (Section 4.1), neural network architectures (Section 4.2), baseline (Section 4.3), training configurations (Section 4.4), and evaluation metrics (Section 4.5), thereby ensuring a thorough validation of the novel approach.

4.1. Selected equations

The performance of the proposed approach is evaluated through the solution of several representative equations that span a range of physical phenomena. These equations are selected for their relevance in various scientific and engineering contexts, allowing for a comprehensive assessment of the method's capability to handle different types of dynamics. The specific equations considered are as follows:

Schrödinger equation A fundamental equation in quantum mechanics that governs the evolution of wave functions over space and time, describing quantum systems.

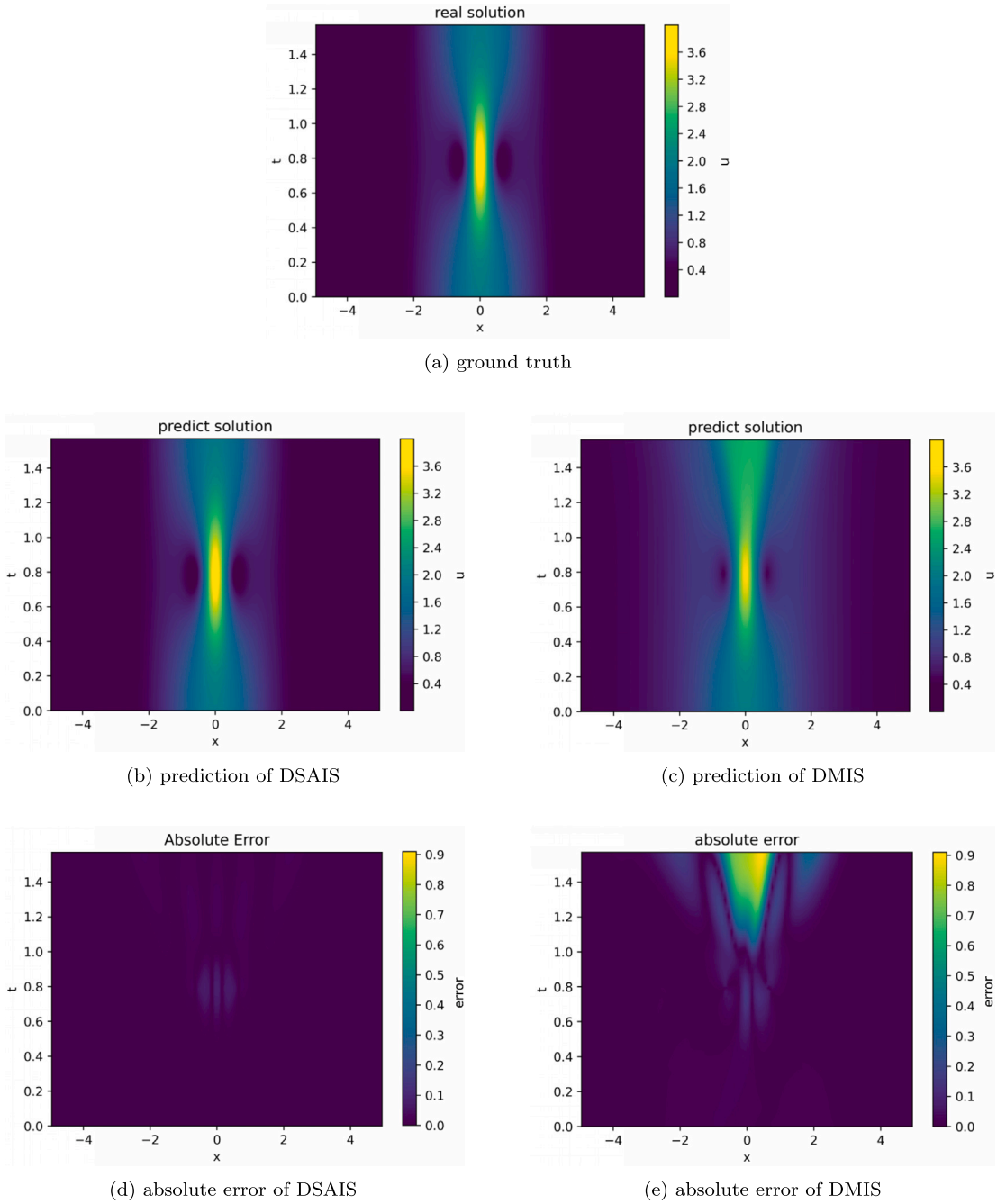
Burgers' equation A nonlinear PDE in fluid dynamics, is often used to model shock waves, turbulence, and other nonlinear advection-diffusion phenomena.

Kortweg-de Vries (KdV) equation A nonlinear PDE that describes solitary waves, or solitons, in one-dimensional systems, frequently applied in fluid mechanics and nonlinear wave theory.

2D Heat equation A PDE describing the diffusion of heat (or temperature variations) over a two-dimensional domain, widely used to model thermal conduction and temperature distribution.

4.2. Neural network architectures

For each equation, a feedforward neural network with fully connected layers is utilized to approximate the solutions. This approach



(a) ground truth

(b) prediction of DSAIS

(c) prediction of DMIS

(d) absolute error of DSAIS

(e) absolute error of DMIS

Fig. 3. The (a) ground truth, (b,c) predictions, and (d,e) absolute errors of solving the Schrödinger equation with PINN-DSAIS and PINN-DMIS.

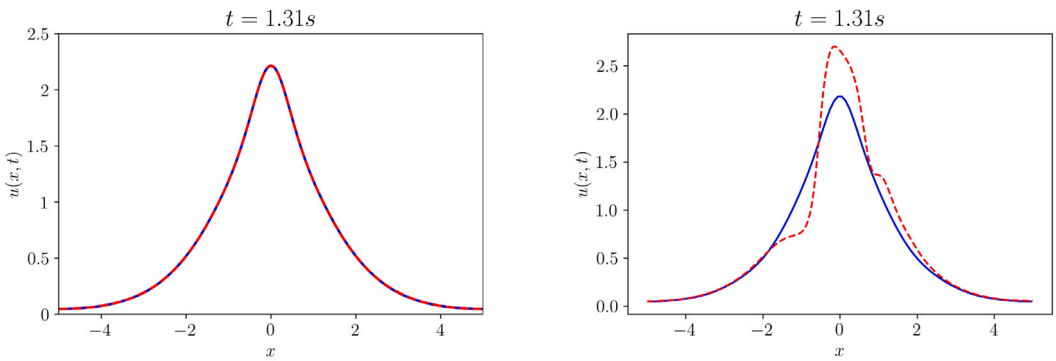
ensures flexibility and adaptability across the different PDEs under consideration. The architecture of the neural network is designed to effectively capture the underlying dynamics of the equations, with specific configurations tailored to the requirements of each task. The configuration details are outlined as follows:

Input layer The number of neurons in this layer corresponds to the dimensions of the independent variables of the equation, such as spatial and temporal coordinates. For instance, in the case of the Schrödinger equation, the inputs would include both spatial and temporal variables, ensuring that the network can effectively model the evolution of the wave function over time and space.

Hidden layers The network includes fully connected layers with a varying number of neurons, which is determined by the complexity of the specific equation. The number of neurons per layer is adjusted to allow the network to capture the nonlinearities inherent in the equation. Nonlinear activation functions, such as ReLU or tanh, are applied to introduce necessary nonlinearity, enabling the network to model complex behaviors and interactions present in the equations.

Output layer The number of neurons in the output layer is chosen to match the dimensions of the solution space. For example, in the case of the Schrödinger equation, this would involve separate neurons for both the real and imaginary components of the wave function. This structure ensures that the network can represent the full solution, whether it is a scalar field, vector field, or more complex representation.

By tailoring the network architecture to the characteristics of each equation, the proposed method is expected to effectively capture the



(a) PINN-DSAIS (b) PINN-DMIS

Fig. 4. Comparison between the Schrödinger equation results (red) solved by (a) PINN-DSAIS and (b) PINN-DMIS and the ground truth (blue).

relevant dynamics and provide accurate approximations of the solutions.

4.3. Baseline

To assess the performance of the proposed method, a series of comparisons are performed against several established baseline approaches. These benchmarks represent a range of existing methods in the field of PINNs and provide a comprehensive framework for evaluating the effectiveness of the proposed technique in terms of both accuracy and computational efficiency. The baseline methods selected for comparison are as follows:

PINN-O The original PINN method [14], which forms the foundation of the approach by incorporating physical laws into the loss function to guide the training process.

PINN-N A variant of PINN that integrates importance sampling techniques to enhance the accuracy and robustness of the solution [34], particularly when dealing with complex or noisy data.

xPINN An extension of PINN designed specifically for domain decomposition [38], which allows for more efficient handling of large-scale problems by dividing the domain into smaller subdomains that can be processed in parallel.

cPINN A conservative variant of PINN that focuses on preserving the conservation properties of physical quantities [39], such as mass, momentum, or energy, during the training process.

PINN-DMIS An enhanced version of PINN that incorporates dynamic importance sampling [21], a technique aimed at improving convergence and accuracy by adapting the sampling strategy based on the evolving solution.

These baseline methods are chosen to highlight the advancements introduced by the proposed approach, particularly in terms of its ability to improve solution accuracy, reduce computational overhead, and handle a broader range of complex problem settings. By comparing the proposed method against these established models, the study aims to demonstrate its advantages and contributions to the field.

4.4. Training configurations

To evaluate the effectiveness of the training strategy for the proposed method, several configurations are employed, each chosen to optimize specific aspects of the training process. These configurations include the selection of optimizers, learning rates, and collocation point strategies, which are designed to balance efficiency and accuracy in solving the targeted partial differential equations. The training configurations are as follows:

Optimizers To ensure efficient gradient-based updates, the Adam optimizer is used during the initial phase of training, covering the first half of the total epochs. Adam has been shown to accelerate convergence in high-dimensional problems [40]. For the fine-tuning phase, the L-BFGS optimizer is employed, taking advantage of its quasi-Newton properties to provide rapid convergence with high precision [41].

Learning Rate The learning rates for each optimizer are set to specific values to optimize convergence. For Adam, initial learning rates are typically in the range of 1×10^{-4} to 1×10^{-5}, based on prior applications. For L-BFGS, adaptive learning rates are used to maintain stability during fine-tuning and ensure smooth convergence.

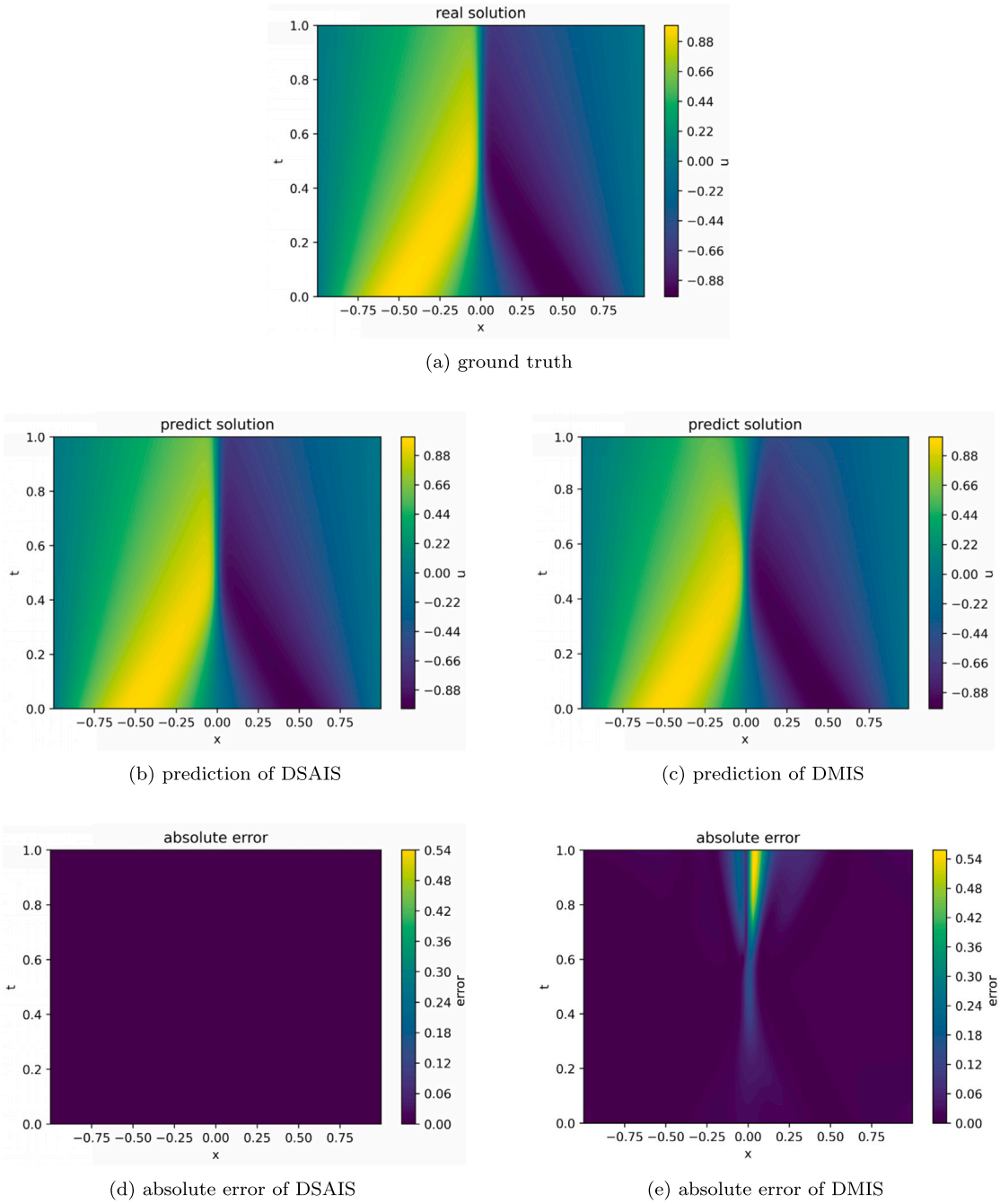
Collocation Points To enforce the PDE constraints, collocation points are initially sampled within the domain using the Latin Hypercube Sampling (LHS) method. Additionally, Voronoi-based dynamic sampling is introduced to enhance the point distribution near regions of interest, such as areas with sharp gradients or physical phenomena of particular importance. The number of initial seed points ranges from 500 to 1000, depending on the complexity of the equation being solved. As training advances, up to 2500 additional seed points are dynamically allocated to enhance the solution. This is achieved by randomly determining the number of required points within each cell and updating the additional seeds every 10 epochs, thereby ensuring improved coverage of critical regions. It is important to note that the time spent on updating these additional seed points accounts for roughly half of the training time per epoch.

These configurations are chosen to ensure that the training process is both efficient and effective in capturing the underlying dynamics of the equations being solved. By carefully selecting and tuning these elements, the proposed method is expected to achieve high accuracy while maintaining computational efficiency.

4.5. Evaluation metrics

To comprehensively assess the effectiveness of the proposed method, three key performance metrics are employed, each serving a distinct purpose in evaluating different aspects of the solution's accuracy. These metrics provide a balanced overview of the model's ability to approximate the ground truth while capturing both the overall error distribution and the impact of larger deviations. The selected metrics are as follows:

Maximum Error (ME) This metric quantifies the worst-case deviation between the predicted solution and the ground truth, providing insight into the largest discrepancy encountered across the domain. It is particularly useful for identifying regions where the model may fail to accurately approximate the solution.



(a) ground truth

(b) prediction of DSAIS

(c) prediction of DMIS

(d) absolute error of DSAIS

(e) absolute error of DMIS

Fig. 5. The (a) ground truth, (b,c) predictions, and (d,e) absolute errors of solving the Burgers equation with PINN-DSAIS and PINN-DMIS.

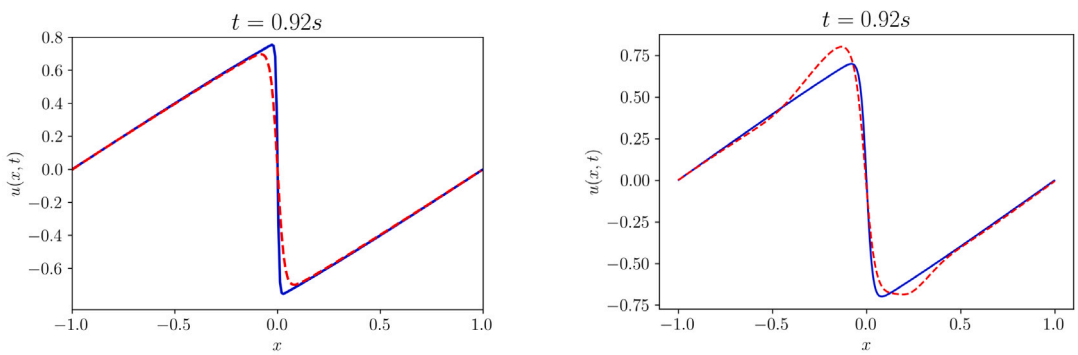
Mean Absolute Error (MAE) The MAE measures the average absolute error over the entire domain, offering a general indication of the model's overall accuracy. Unlike metrics that involve squared terms, MAE provides a direct, interpretable measure of the average deviation from the true values, making it suitable for assessing the general performance across the solution space.

Root Mean Square Error (RMSE) This metric computes the square root of the mean of the squared errors, emphasizing larger deviations more heavily than smaller ones. RMSE is particularly useful for capturing significant errors and assessing how well the model performs in regions where the solution may experience sharp gradients or rapid changes.

By using these three complementary metrics, the proposed method is thoroughly evaluated from different perspectives, ensuring a comprehensive understanding of its performance across various solution characteristics.

5. Results and analysis

In this section, the results and analysis of the proposed PINN-DSAIS framework are presented across four PDEs. Section 5.1 evaluates the Schrödinger equation, demonstrating accurate wave dynamics modeling. Section 5.2 focuses on the Burgers' equation, highlighting robustness in resolving shock waves. Section 5.3 examines the KdV



(a) PINN-DSAIS (b) PINN-DMIS

Fig. 6. Comparison between the Burgers equation results (red) solved by (a) PINN-DSAIS and (b) PINN-DMIS and the ground truth (blue).

equation, showcasing the method's capability in soliton dynamics. Section 5.4 presents the results for the 2D heat equation, demonstrating that the method's framework effectively captures diffusion-dominated thermal behavior with high accuracy and efficiency. Quantitative and qualitative analyses validate the method's effectiveness throughout.

5.1. Schrödinger equation

The Schrödinger equation is a fundamental quantum mechanics equation that governs non-relativistic particles' behavior. It describes the evolution of wave functions over time and space, serving as one of the cornerstone laws of quantum mechanics. With the IC of $u(0,x) = 2sech(x)$ and periodic BCs, this equation can be expressed as:

$$i\frac{\partial u}{\partial t} + 0.5\frac{\partial^2 u}{\partial x^2} + |u|^2 u = 0, \quad x \in [-5,5], \quad t \in [0,\pi/2] \tag{10}$$

As previously stated, all results were obtained using a fully connected neural network. The network architecture consists of an input layer with 2 neurons, corresponding to the x and t inputs, followed by 5 hidden layers, each containing 100 neurons, and an output layer with 2 neurons representing the real and imaginary components. The training was performed over 10,000 epochs: the first 5000 epochs utilized the Adam optimizer with a learning rate of 0.001. In comparison, the final 5000 epochs employed the L-BFGS optimizer with a learning rate 1.0. The Voronoi diagram was initially constructed using 1000 seed points, and 2000 more seed points were later added. The numbers of points selected for each condition were as follows: $\mathcal{T}_i = 100$ for initial conditions, $\mathcal{T}_b = 100$ for boundary conditions, and $\mathcal{T}_f = 50{,}000$ for collocation points. From these 50,000 collocation points, a mini-batch of 20,000 samples was selected for training using importance sampling.

The results presented in Fig. 3(a)–(e) exhibit a high degree of consistency between the solution obtained using the PINN-DSAIS method and the ground truth. The absolute error distribution reveals that the error is primarily concentrated in the region near x = 0, with overall error magnitudes remaining low. This indicates that the network achieves high accuracy in learning the dynamic evolution of the solution to the Schrödinger equation. However, the localized concentration of error suggests that the model's fitting capability in these regions could still be further refined, particularly where the wave function exhibits complex nonlinear behavior.

In comparison, as shown in Fig. 4(a) and (b), the PINN-DSAIS method provides predictions at $t = 1.31$ s that almost perfectly align with the ground truth, while the PINN-DMIS method exhibits noticeable deviations at the same time point. Specifically, significant discrepancies are observed around $x = -2$ and $x = 2$ for the PINN-DMIS method. These results highlight the effectiveness of DSAIS in dynamically allocating sampling points, which significantly increases the sampling density in critical regions, thereby enhancing the model's learning capability.

Meanwhile, Table 1 comprehensively compares the performance of various methods applied to the Schrödinger equation, Burgers' equation, and the KdV equation. From a quantitative perspective, the PINN-DSAIS method achieves an MAE of 0.005 and an RMSE of 0.0113, whereas the PINN-DMIS method results in higher errors, with an MAE of 0.127 and an RMSE of 0.2196. This significant difference further verifies the superiority of DSAIS in solving nonlinear and complex equations.

Table 2 illustrates the results of an ablation study analyzing the influence of different initial seed point counts and additional seed allocations on the performance of the model for the Schrödinger equation, Burgers' equation, and KdV equation. For the Schrödinger equation, the results reveal that increasing the number of initial and extra seed points can marginally enhance model accuracy. The configuration with 1000 initial seeds and 2000 extra seeds achieves the lowest errors across all metrics, with an ME of 0.0860, an MAE of 0.0052, and an RMSE of 0.0113. These findings suggest that higher seed point densities allow the model to better capture the complex dynamics of the wave function by improving the resolution of critical regions, particularly near boundaries and high-gradient areas. However, the performance improvement is relatively small compared to configurations with fewer seed points, such as 500 initial seeds and 2500 extra seeds, indicating that the model exhibits diminishing returns when seed points exceed a certain threshold.

5.2. Burgers' equation

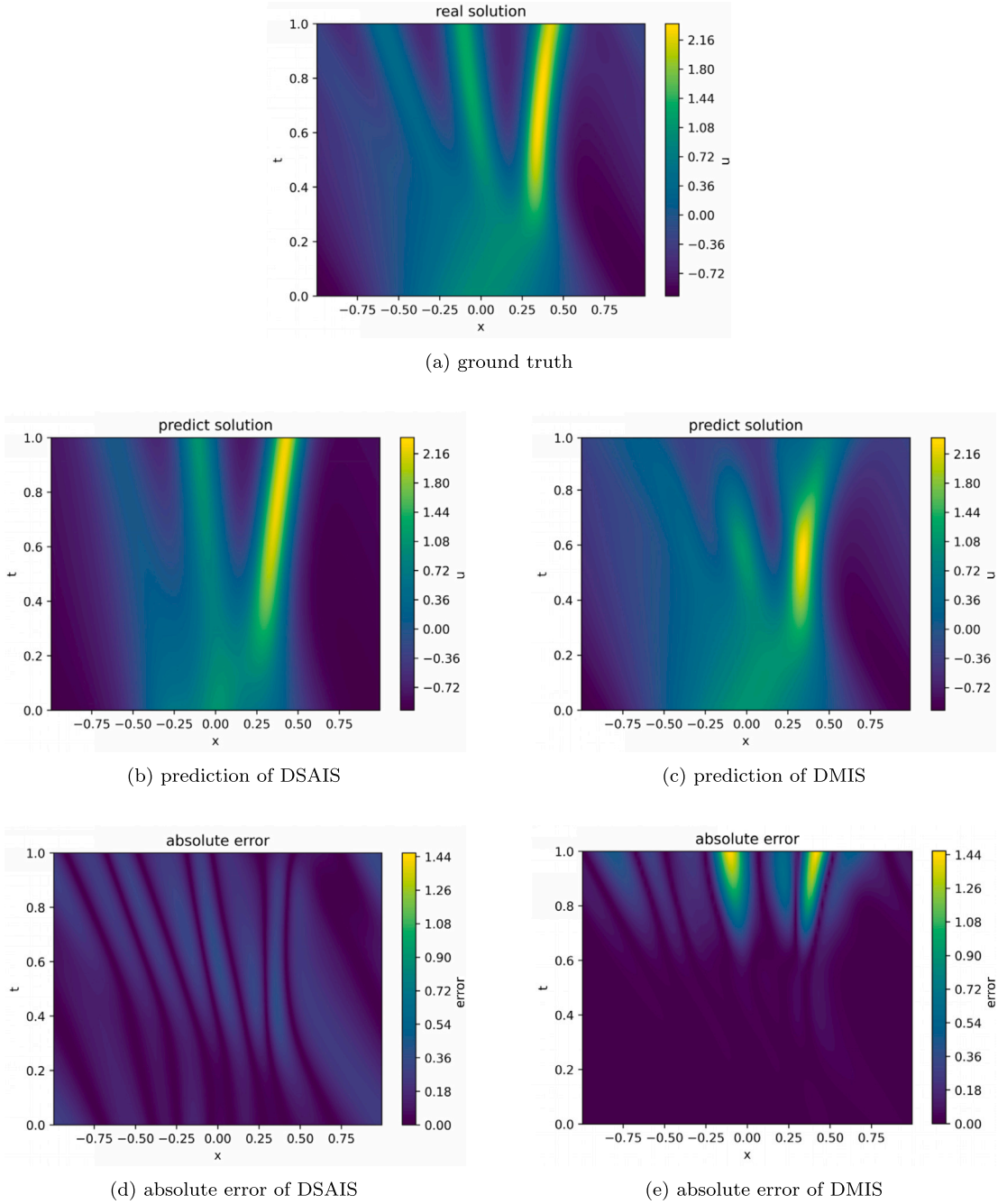
The Burgers' equation is a fundamental PDE in fluid dynamics and nonlinear wave theory. It characterizes the propagation of shock waves and incorporates both nonlinear convection and diffusion terms. This equation, with the IC of $u(0,x) = -\sin(\pi x)$ and the BC of $u(t,x) = 0$ for $x \in \{-1,1\}$, can be expressed as:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{0.04}{\pi}\frac{\partial^2 u}{\partial x^2}, \quad x \in [-1,1], \quad t \in [0,1] \tag{11}$$

In this case, the network architecture consists of 7 layers, each with 20 neurons. The training process spanned 10,000 epochs, with the first 5000 epochs utilizing the Adam optimizer at a learning rate of 1×10^{-4}, followed by another 5000 epochs using the L-BFGS optimizer with a learning rate of 1.0. For constructing the Voronoi diagram, an initial set of 500 seed points was partitioned, supplemented by an additional 2500 seed points. Within the domain, 100 boundary points and 100 initial points were selected, along with 60,000 collocation points. From these collocation points, a mini-batch of 10,000 samples was selected for training using

importance sampling.

Fig. 5(a)–(e) demonstrates that the PINN-DSAIS method effectively captures the dynamics of the Burgers' equation, with the solution closely aligning with the ground truth. The absolute error distribution



(a) ground truth

(b) prediction of DSAIS

(c) prediction of DMIS

(d) absolute error of DSAIS

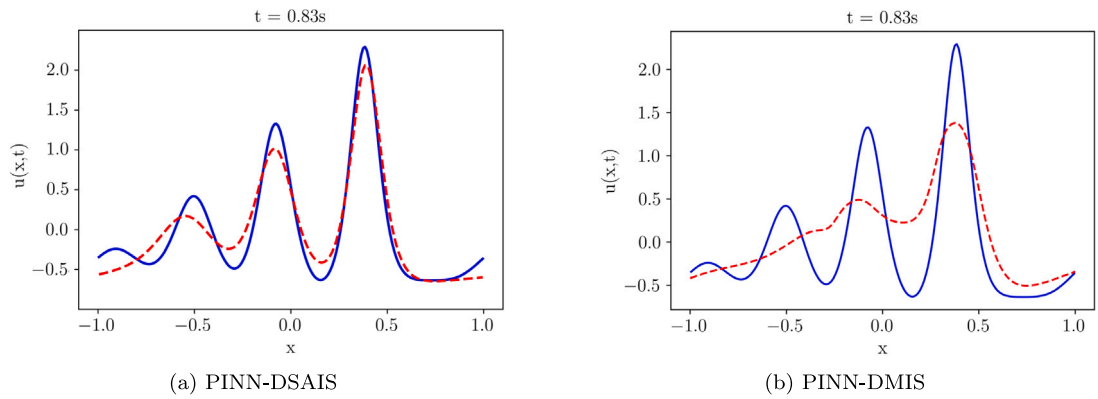
(e) absolute error of DMIS

Fig. 7. The (a) ground truth, (b,c) predictions, and (d,e) absolute errors of solving the KdV equation with PINN-DSAIS and PINN-DMIS.

Table 1
Comparison of methods for solving Schrödinger, Burgers, and KdV equations.

Method	Schrödinger equation			Burgers equation			KdV equation		
	ME	MAE	RMSE	ME	MAE	RMSE	ME	MAE	RMSE
PINN-O	1.360	0.186	0.4092	0.451	0.0738	0.1100	2.140	0.363	0.520
PINN-N	0.948	0.149	0.2906	0.358	0.0579	0.0859	1.860	0.292	0.441
xPINN	0.546	0.045	**0.0089**	0.261	0.0099	0.0010	2.462	0.272	0.230
cPINN	0.591	0.069	0.0169	0.324	0.0084	**0.0007**	2.925	0.258	0.248
PINN-DMIS	0.647	0.127	0.2196	0.225	0.0294	0.0495	1.170	0.391	0.492
PINN-DSAIS	**0.086**	**0.005**	0.0113	**0.010**	**0.0011**	0.0016	**0.396**	**0.133**	**0.160**

shows low error magnitudes overall, with minor concentrations near the discontinuity, indicating the model's robustness in handling shock waves. In Fig. 6(a) and (b), a comparison of PINN-DSAIS and PINN-DMIS further highlights the superiority of DSAIS. At $t = 0.92$ s, PINN-DSAIS achieves near-perfect agreement with the ground truth, while PINN-DMIS shows visible deviations, particularly around the shock region. This improvement can be attributed to DSAIS's dynamic sampling strategy, which allocates more points to critical regions,



(a) PINN-DSAIS (b) PINN-DMIS

Fig. 8. Comparison between the KdV equation results (red) solved by (a) PINN-DSAIS and (b) PINN-DMIS and the ground truth (blue).

Table 2
Comparison of experimental results of Schrödinger equation, Burgers equation, and KdV equation under different numbers of seed points.

Initial seeds	Extra seeds	Schrödinger equation			Burgers' equation			KdV equation		
		ME	MAE	RMSE	ME	MAE	RMSE	ME	MAE	RMSE
500	1500	0.0883	0.0054	0.0114	0.0137	0.0018	0.0026	0.3866	0.1347	0.1608
1000	1000	0.0895	0.0054	0.0114	0.0241	0.0025	0.0036	0.3854	0.1351	0.1612
500	2500	0.0863	0.0053	0.0113	**0.0096**	**0.0011**	**0.0016**	0.3958	**0.1334**	**0.1598**
1000	2000	**0.0860**	**0.0052**	**0.0113**	0.0245	0.0021	0.0038	**0.3832**	0.1349	0.1611

thereby enhancing the model's ability to resolve steep gradients and nonlinear features.

For the Burgers' equation, PINN-DSAIS achieves the lowest ME of 0.010 and MAE of 0.0011, indicating its ability to accurately capture both the nonlinear convection and diffusion dynamics. In Table 2, a similar trend is observed in the results, where optimal performance is achieved with 500 initial seeds and 2500 additional seeds, producing the lowest ME of 0.0096, MAE of 0.0011, and RMSE of 0.0016. This outcome highlights the importance of balancing initial seed point allocation with the addition of extra seeds, as an excessive concentration of seed points may lead to overfitting in smooth regions while neglecting more complex dynamics.

5.3. Korteweg–de Vries equation

The Korteweg–de Vries (KdV) equation is a nonlinear PDE widely used to describe wave propagation in one-dimensional incompressible fluids. It holds significant applications in mathematics, physics, and engineering, particularly in the study of nonlinear waves, integrable systems, soliton theory, and dynamical systems. In this study, the KdV equation is considered with the IC of $u(0,x) = \cos(\pi x)$ and periodic BC, expressed as:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + 0.0025\frac{\partial^3 u}{\partial x^3} = 0, \quad x \in [-1,1], \quad t \in [0,1] \tag{12}$$

The network architecture used in this experiment comprises an input layer with 2 neurons corresponding to the x and t inputs, followed by 7 hidden layers, each with 64 neurons, and an output layer with a single neuron. The number of collocation, initial, and boundary points is set as $\mathcal{T}_f = 60{,}000$, $\mathcal{T}_i = 200$, and $\mathcal{T}_b = 200$, respectively. From the 60,000 collocation points, 20,000 points are selected for training by importance sampling. The learning rates for the Adam and L-BFGS optimizers are 1×10^{-5} and 0.8, respectively.

Fig. 7(a)–(e) confirms the effectiveness of the PINN-DSAIS method in solving the KdV equation. The solution aligns closely with the ground truth, while the absolute error remains relatively low across the domain. Minor error concentrations are observed in regions with steep gradients, reflecting the inherent challenges of capturing nonlinear wave propagation. In Fig. 8(a) and (b), comparing PINN-DSAIS with PINN-DMIS reveals the precedence of DSAIS, particularly in resolving soliton dynamics. At $t = 0.83$ s, PINN-DSAIS predictions are nearly indistinguishable from the ground truth, while PINN-DMIS exhibits noticeable deviations in the amplitudes and positions of the solitons. This improvement highlights the advantage of dynamically allocated sampling points in accurately capturing the complex behavior of nonlinear waves.

For the KdV equation, PINN-DSAIS yields the lowest ME of 0.396, MAE of 0.133, and RMSE of 0.160, outperforming traditional methods such as PINN-O and PINN-N, as well as advanced methods like xPINN and cPINN. However, the results exhibit slightly higher overall errors compared to the other equations due to the more intricate nonlinear interactions present in the solution. The configuration with 1000 initial seeds and 1000 extra seeds achieves the lowest MAE of 0.1347, while the configuration with 500 initial seeds and 2500 extra seeds shows competitive performance in terms of ME and RMSE. These findings underline the need to distribute seed points in highly nonlinear phenomena models carefully.

5.4. 2D heat equation

In addition to the three representative 1D transient PDEs discussed above, another issue of great interest is whether the computational cost of the novel approach increases significantly with increasing dimensionality. In response, a two-dimensional spatial example is included here to demonstrate the scalability and efficiency of the proposed method. The 2D heat equation is a fundamental PDE that describes the diffusion of heat or temperature variations across a spatial domain over time. It is extensively applied in engineering and physics to model thermal conduction and temperature distribution in various materials. In this study, we consider the 2D heat equation with the initial condition:

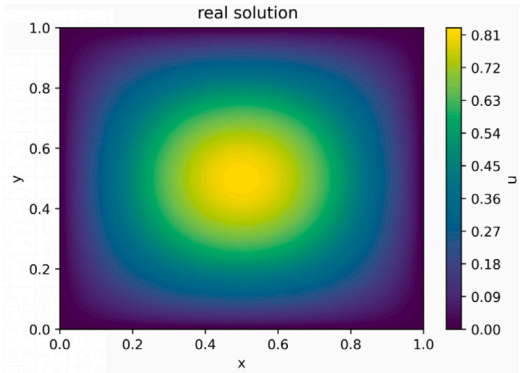
$$u(x,y,0) = \sin(\pi x)\sin(\pi y) \tag{13}$$

and Dirichlet boundary conditions:

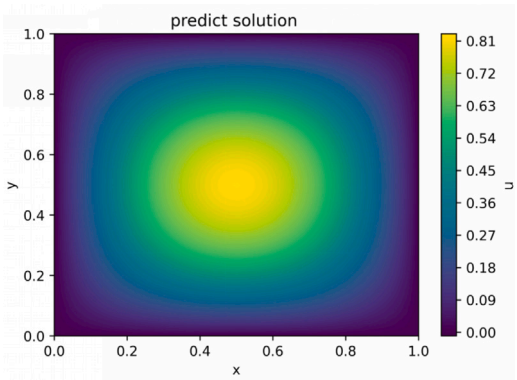
$$u(x,y,t) = 0 \quad \text{on} \quad x = 0,\ x = 1,\ y = 0,\ y = 1, \tag{14}$$

For $(x,y) \in [0,1] \times [0,1]$ and $t \in [0,1]$. The governing equation is expressed as:

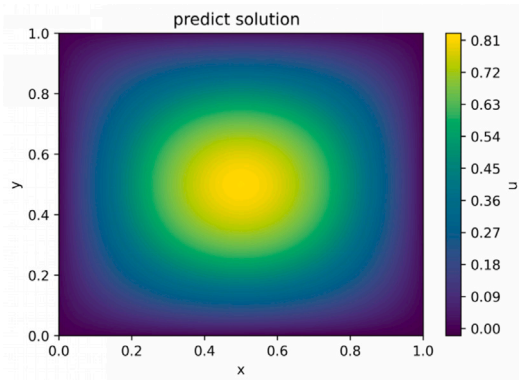
$$\frac{\partial u}{\partial t} - 0.01\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = 0, \tag{15}$$



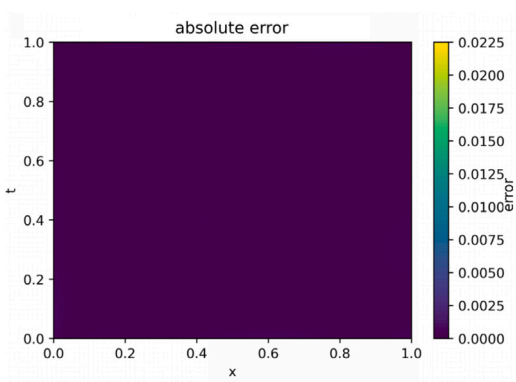
(a) ground truth



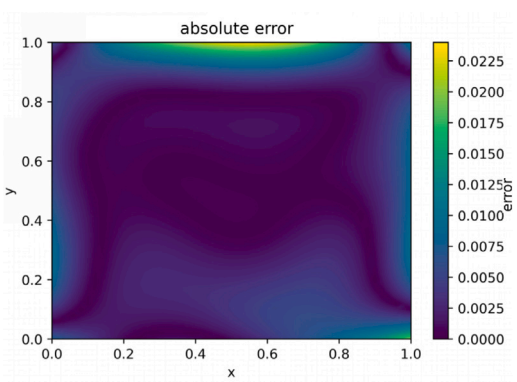
(b) prediction of DSAIS



(c) prediction of DMIS



(d) absolute error of DSAIS



(e) absolute error of DMIS

Fig. 9. The (a) ground truth, (b,c) predictions, and (d,e) absolute errors of solving the 2D Heat equation at the final time with PINN-DSAIS and PINN-DMIS.

The neural network architecture used in this experiment comprises an input layer with 3 neurons corresponding to x, y, and t, followed by 5 hidden layers with 50 neurons each, and an output layer with a single neuron. The number of collocation, initial, and boundary points is set as $\mathcal{T}_f = 60{,}000$, $\mathcal{T}_i = 200$, and $\mathcal{T}_b = 200$, respectively. From the 60,000 collocation points, 10,000 points are selected for training by importance sampling. The learning rates for the Adam and L-BFGS optimizers are 1×10^{-4} and 1.0, respectively. Fig. 9(a)–(e) and Table 3 confirm the effectiveness of the PINN-DSAIS method in solving the 2D heat equation. The solution aligns closely with the ground truth, while the absolute error remains relatively low across the domain. Minor error concentrations are observed in regions with steep gradients, reflecting the inherent challenges of capturing nonlinear wave propagation.

6. Discussion

To summarize, based on the observation that regions with higher loss naturally warrant greater attention, the proposed method builds upon previous works, PINN-DMIS and PINN-N. Our approach excels by allocating more seed points in these high-loss areas: the loss at these seed points is precisely computed, and subsequently, the loss at other collocation points is interpolated to determine their sampling probabilities. This strategy enhances the accuracy of the interpolated

Table 3
Comparison of methods for solving 2D heat equation.

Method	2D heat equation		
	ME	MAE	RMSE
PINN-O	0.0230	0.0075	0.0085
PINN-N	0.0211	0.0021	0.0030
PINN-DMIS	0.0239	0.0021	0.0031
PINN-DSAIS	**0.0012**	**0.0001**	**0.0001**

loss estimates, ensuring that the computed probabilities closely approximate those obtained through exact calculations. However, the method still has certain room for improvement.

Initially, Voronoi partitioning is required, followed by interpolation within each Voronoi cell, both of which contribute to a high computational complexity. Additionally, the number of seed points chosen for the initial allocation and the total number of seed points added subsequently greatly affect the final interpolation results, and determining these numbers accurately remains a challenging task. Automating the selection of these quantities—perhaps via adaptive stopping criteria or hierarchical seeding schemes—represents a promising direction for reducing complexity. Moreover, exploring alternative interpolation kernels, coupling DSAIS with residual-based error estimators, or integrating mesh-free radial basis functions may further enhance efficiency and robustness. These avenues offer fertile ground for future research aimed at balancing sampling cost against accuracy gains in PINN training.

7. Conclusion

In this article, we introduce a novel algorithm called Dynamic Seed Allocation Importance Sampling (DSAIS), which builds upon the principles of importance sampling. The key innovation of DSAIS lies in its ability to allocate seed points adaptively in regions of high error, enabling a more accurate estimation of sampling probabilities. These probabilities are then efficiently interpolated to determine the sampling likelihood of other collocation points. The experimental evaluation, conducted on three widely recognized benchmark equations, demonstrates that DSAIS significantly outperforms existing importance sampling methods, particularly in terms of solution accuracy. Notably, when applied to the highly nonlinear Schrödinger equation, the new approach achieves a root mean square error reduction of up to 63%, highlighting its robust performance in challenging scenarios.

Despite the substantial advancements offered by DSAIS, we acknowledge that further improvements are possible. Critical factors such as the optimal selection of initial seed points, the strategic distribution of additional seed points within each Voronoi cell, and the choice of an effective interpolation strategy all play pivotal roles in influencing the overall performance. Addressing these aspects could lead to even greater enhancements in efficiency and accuracy. Overall, the significant improvement in solution accuracy achieved by DSAIS underscores its potential as a promising direction for further research and optimization in the field of importance sampling and PDE solvers.

CRediT authorship contribution statement

Wentao Feng: Writing – review & editing, Supervision, Funding acquisition. **Ming Hu:** Writing – original draft, Methodology, Investigation, Conceptualization. **Rui Huang:** Conceptualization. **Chenwei Tang:** Supervision. **Shudong Huang:** Writing – review & editing, Supervision. **Jiancheng Lv:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62306199, 62376175).

Data availability

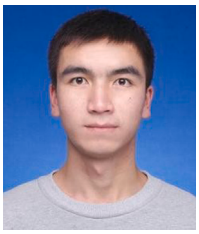
No data was used for the research described in the article.

References

[1] Muhammad Akbar, Rashid Nawaz, Sumbal Ahsan, Kottakkaran Sooppy Nisar, Kamal Shah, Emad E Mahmoud, MM Alqarni, Fractional power series approach for the solution of fractional-order integro-differential equations, Fractals (2022).
[2] John R Cannon, The solution of the heat equation subject to the specification of energy, Quart. Appl. Math. (1963).
[3] Alexandre Joel Chorin, Numerical solution of the Navier–Stokes equations, Math. Comp. (1968).
[4] Muhammad Sher, Kamal Shah, Zareen A Khan, Hasib Khan, Aziz Khan, Computational and theoretical modeling of the transmission dynamics of novel COVID-19 under Mittag-Leffler power law, Alex. Eng. J. (2020).
[5] Rohul Amin, Kamal Shah, Qasem M Al-Mdallal, Imran Khan, Muhammad Asif, Efficient numerical algorithm for the solution of eight order boundary value problems by haar wavelet method, Int. J. Appl. Comput. Math. (2021).
[6] Tor Steinar Schei, A finite-difference method for linearization in nonlinear estimation algorithms, Automatica (2020).
[7] Philippe G Ciarlet, Finite volume methods and their analysis, SIAM (2002).
[8] Jan S Hesthaven, Sigal Gottlieb, David Gottlieb, Spectral Methods for Time-Dependent Problems, Cambridge University Press, 2007.
[9] Xin-Yang Liu, Min Zhu, Lu Lu, Hao Sun, Jian-Xun Wang, Multi-resolution partial differential equations preserved learning framework for spatiotemporal dynamics, Commun. Phys. (2024).
[10] Justin Sirignano, Konstantinos Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, J. Comput. Phys. (2018).
[11] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, Kenji Kawaguchi, Tackling the curse of dimensionality with physics-informed neural networks, Neural Netw. (2024).
[12] Kurt Hornik, Maxwell Stinchcombe, Halbert White, Multilayer feedforward networks are universal approximators, Neural Netw. (1989).
[13] Zheyuan Hu, Ameya D Jagtap, George Em Karniadakis, Kenji Kawaguchi, When do extended physics-informed neural networks (XPINNs) improve generalization? SIAM J. Sci. Comput. (2022).
[14] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. (2019).
[15] Sifan Wang, Hanwen Wang, Paris Perdikaris, On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. (2021).
[16] Shuai Han, Lukas Stelz, Horst Stoecker, Lingxiao Wang, Kai Zhou, Approaching epidemiological dynamics of COVID-19 with physics-informed neural networks, J. Franklin Inst. (2024).
[17] Haoran Hu, Connor M Kennedy, Panayotis G Kevrekidis, Hong-Kun Zhang, A modified PINN approach for identifiable compartmental models in epidemiology with application to COVID-19, Viruses (2022).
[18] Xuhui Meng, Zhen Li, Dongkun Zhang, George Em Karniadakis, PPINN: Parareal physics-informed neural network for time-dependent PDEs, Comput. Methods Appl. Mech. Engrg. (2020).
[19] Gregory Kang Ruey Lau, Apivich Hemachandra, See-Kiong Ng, Bryan Kian Hsiang Low, PINNACLE: PINN adaptive ColLocation and experimental points selection, in: NeurIPS 2023 Workshop on Adaptive Experimental Design and Active Learning in the Real World, 2023.
[20] Namgyu Kang, Byeonghyeon Lee, Youngjoon Hong, Seok-Bae Yun, Eunbyung Park, Pixel: Physics-informed cell representations for fast and accurate pde solvers, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2023.
[21] Zijiang Yang, Zhongwei Qiu, Dongmei Fu, Dmis: Dynamic mesh-based importance sampling for training physics-informed neural networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2023.
[22] Shudong Huang, Wentao Feng, Chenwei Tang, Zhenan He, Caiyang Yu, Jiancheng Lv, Partial differential equations meet deep neural networks: A survey, IEEE Trans. Neural Netw. Learn. Syst. (2025).
[23] Alexander Kovacs, Lukas Exl, Alexander Kornell, Johann Fischbacher, Markus Hovorka, Markus Gusenbauer, Leoni Breth, Harald Oezelt, Masao Yano, Noritsugu Sakuma, et al., Conditional physics informed neural networks, Commun. Nonlinear Sci. Numer. Simul. (2022).
[24] Arka Daw, Jie Bu, Sifan Wang, Paris Perdikaris, Anuj Karpatne, Rethinking the importance of sampling in physics-informed neural networks, 2022, arXiv preprint arXiv:2207.02338.

[25] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, Lu Lu, A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. (2023).
[26] Zhiyuan Zhao, Xueying Ding, B Aditya Prakash, Pinnsformer: A transformer-based framework for physics-informed neural networks, 2023, arXiv preprint arXiv:2307.11833.
[27] Yihang Gao, Michael K Ng, Wasserstein generative adversarial uncertainty quantification in physics-informed neural networks, J. Comput. Phys. (2022).
[28] Pu Ren, Chengping Rao, Yang Liu, Jian-Xun Wang, Hao Sun, PhyCRNet: Physics-informed convolutional-recurrent network for solving spatiotemporal PDEs, Comput. Methods Appl. Mech. Engrg. (2022).
[29] Levi D McClenny, Ulisses M Braga-Neto, Self-adaptive physics-informed neural networks, J. Comput. Phys. (2023).
[30] Sifan Wang, Yujun Teng, Paris Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. (2021).
[31] Reuven Y Rubinstein, Dirk P Kroese, Simulation and the Monte Carlo Method, John Wiley & Sons, 2016.
[32] Dominik Csiba, Peter Richtárik, Importance sampling for minibatches, J. Mach. Learn. Res. (2018).
[33] Angelos Katharopoulos, François Fleuret, Not all samples are created equal: Deep learning with importance sampling, in: International Conference on Machine Learning, 2018.
[34] Mohammad Amin Nabian, Rini Jasmine Gladstone, Hadi Meidani, Efficient training of physics-informed neural networks via importance sampling, Comput.-Aided Civ. Infrastruct. Eng. (2021).
[35] Deanna Needell, Rachel Ward, Nati Srebro, Stochastic gradient descent, weighted sampling, and the randomized Kaczmarz algorithm, Adv. Neural Inf. Process. Syst. (2014).
[36] Peilin Zhao, Tong Zhang, Stochastic optimization with importance sampling for regularized loss minimization, in: International Conference on Machine Learning, 2015.
[37] Angelos Katharopoulos, François Fleuret, Biased importance sampling for deep neural network training, 2017, arXiv preprint arXiv:1706.00043.
[38] Ameya D Jagtap, George Em Karniadakis, Extended physics-informed neural networks (XPINNs): A generalized space–time domain decomposition based deep learning framework for nonlinear partial differential equations, Commun. Comput. Phys. (2020).
[39] Ameya D Jagtap, Ehsan Kharazmi, George Em Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, Comput. Methods Appl. Mech. Engrg. (2020).
[40] Diederik P Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
[41] Jorge Nocedal, Stephen Wright, Numerical Optimization, in: Springer Series in Operations Research and Financial Engineering, 2006.

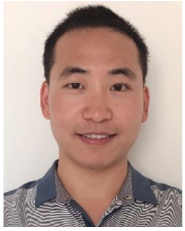
Wentao Feng received his doctoral degree (Dr.-Ing.) in geothermal engineering from the Clausthal University of Technology (TU Clausthal), Clausthal-Zellerfeld, Germany, in 2019. He is currently an associate professor with the College of Computer Science, Sichuan University, Chengdu, China. His research interests include artificial intelligence, numerical simulation, machine learning, deep learning, industrial intelligence, and smart energy.



Ming Hu received his B.Sc. degree in data science and big data technology from Shanghai University of Engineering Science (SUES), Shanghai, China, in 2023. He is currently a M.Sc. student with the College of Computer Science, Sichuan University.

Rui Huang received his B.Sc. degree in software engineering from Hefei University of Technology (HFUT), Hefei, China, in 2023. He is currently a M.Sc. student with the College of Computer Science, Sichuan University.

Chenwei Tang received the Ph.D. degree in computer science and technology from Sichuan University (SCU), Chengdu, China, in 2020. She is currently an associate professor with the College of Computer Science, Sichuan University. Her research interests include deep learning, industrial intelligence, zero-shot learning, multimodal learning, and computer vision.



Shudong Huang received his Ph.D. degree in computer science and engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China. He is currently an associate research professor with the College of Computer Science, Sichuan University, Chengdu, China. His research interests include machine learning, deep learning, pattern recognition and data mining.

Jiancheng Lv received the Ph.D. degree in computer science and engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2006. He was a Research Fellow with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore. He is currently a Professor and Dean of the College of Computer Science, Sichuan University, Chengdu, China. His research interests include neural networks, machine learning, and big data. He has published over 80 research papers in top-tier conferences and journals.